

Faculdade Multivix – Serra/ES
Curso de Engenharia da Computação

Trabalho I

Programação Distribuída e Paralela

Sistema de diálogo entre personagens de RPG com
Filas, Tasks e métodos síncronos/assíncronos

Integrantes: Luiz Fernando Pereira Rojas dos Santos
Tadeu Augusto Vilela da Silva

Curso: Engenharia da Computação

Turma: 2026.2

Professor: Jacques Oliveira

Instituição: Multivix Serra – Serra, ES

Serra – ES, 8 de outubro de 2026

1 Objetivo do trabalho

O trabalho solicitado pelo professor Jacques Oliveira consiste em implementar um sistema simples de diálogo entre dois personagens no estilo RPG, de forma caractere a caractere, utilizando os seguintes recursos da linguagem C#:

- **Filas** (`Queue<T>`) para armazenar as falas de cada personagem;
- **Métodos síncronos e assíncronos** para comparar os dois modelos de execução;
- **Tasks** (`Task`, `async`, `await`) para observar, na prática, o comportamento de operações concorrentes e paralelas e o controle de sincronização entre elas;
- **Orientação a Objetos** (classe, herança e instância), por se tratar de um exercício também de programação distribuída.

2 Repositório do código

O código-fonte completo está publicado, de forma pública, no GitHub:

<https://github.com/LuizRojas/trabalho-programacao-paralela>

O projeto é uma aplicação console em **.NET** e pode ser executada com:

```
dotnet run
```

3 O que foi implementado

3.1 Modelagem orientada a objetos

Foi criada uma classe base `Personagem`, responsável por tudo o que é comum a qualquer personagem do diálogo: nome, tipo, a fila de falas e os métodos de “falar” (síncrono e assíncrono). As classes `Mago` e `Elfo` herdam de `Personagem` e, no construtor, apenas enfileiram as falas específicas de cada uma. Cada personagem usado no programa (`Mago("Gandalf")` e `Elfo("Link")`) é uma **instância** independente dessas classes, com sua própria fila de mensagens.

```
1 public class Personagem
2 {
3     public string nome { get; }
4     public string tipo { get; }
5     protected Queue<string> mensagens;
6     ...
7 }
8
9 public class Mago : Personagem { ... } // herança
10 public class Elfo : Personagem { ... } // herança
```

3.2 Filas (`Queue<T>`)

Cada personagem guarda suas falas em uma `Queue<string>`. A fila garante que as mensagens sejam exibidas sempre na ordem em que foram cadastradas (**FIFO** – **F**irst **I**n, **F**irst **O**ut), que é exatamente o comportamento esperado em um diálogo: a primeira fala cadastrada é a primeira a ser “dita”.

3.3 Método síncrono

O método `DialogoSync` retira uma fala da fila e a imprime letra a letra de forma **bloqueante**, usando `Thread.Sleep`. Enquanto uma fala está sendo exibida, a aplicação não consegue fazer mais nada – é a thread principal que fica ocupada do início ao fim da fala.

3.4 Método assíncrono (`async/await/Task`)

O método `DialogoAsync` tem a mesma ideia, mas retorna `Task` e usa `await Task.Delay(...)` em vez de `Thread.Sleep`. Isso faz com que a espera entre cada caractere **não bloqueie** a thread: o controle é devolvido ao chamador, que pode iniciar outras tasks em paralelo enquanto a atual está “aguardando”.

```
1 public async Task DialogoAsync(int tempoFala = 80)
2 {
3     ...
4     await _semaphore.WaitAsync();
5     try
6     {
7         foreach (char c in msn)
8         {
9             Console.Write($"{c}");
10            await Task.Delay(tempoFala);
11        }
12    }
13    finally
14    {
15        _semaphore.Release();
16    }
17 }
```

3.5 Sincronização com `SemaphoreSlim`

No `Program.cs`, várias chamadas a `DialogoAsync` são disparadas quase ao mesmo tempo e aguardadas com `Task.WhenAll`, caracterizando execução **concorrente** (várias tasks “em andamento” ao mesmo tempo). Se nenhum controle fosse feito, os caracteres de falas diferentes poderiam se misturar no console (ex.: M, de “Mago”, intercalado com E, de “Elfo”).

Para evitar isso, foi usado um `SemaphoreSlim(1, 1)`, que funciona como um cadeado: apenas uma task por vez pode estar dentro da seção que imprime uma fala completa. As demais tasks ficam aguardando (`await _semaphore.WaitAsync()`) até a atual liberar o semáforo (`_semaphore.Release()`), garantindo que cada fala apareça inteira e organizada, mesmo com várias tasks rodando ao mesmo tempo.

3.6 Disparo em paralelo com `Task[]` e `Task.WhenAll`

```
1 Task[] tarefas = {
2     mago.DialogoAsync(600),
3     elfo.DialogoAsync(),
4     mago.DialogoAsync(1000),
5     elfo.DialogoAsync(),
6     mago.DialogoAsync(800),
7     elfo.DialogoAsync()
8 }
```

```
8 };  
9  
10 await Task.WhenAll(tarefas);
```

Todas as tasks são criadas de uma vez (array `Task[]`) e o programa só continua quando todas terminarem (`Task.WhenAll`), demonstrando o paralelismo de forma controlada, em contraste com a execução puramente sequencial do bloco síncrono anterior.

4 Entendimento do grupo

4.1 Concorrência vs. paralelismo

Concorrência é a capacidade de lidar com várias tarefas “ao mesmo tempo” do ponto de vista lógico (elas se alternam, podem estar pausadas aguardando algo). **Paralelismo** é quando essas tarefas realmente executam simultaneamente (em múltiplos núcleos). No trabalho, o uso de `async/await` cria concorrência: as tasks de diálogo ficam alternando entre executar um pedaço e aguardar (`Task.Delay`), sem travar a thread principal – e é o `Task.WhenAll` que efetivamente as executa de forma paralela/concorrente.

4.2 Por que síncrono bloqueia e assíncrono não

No método síncrono, `Thread.Sleep` realmente “congela” a thread até o tempo passar. No método assíncrono, `await Task.Delay` devolve o controle para quem chamou a task, permitindo que outras tasks avancem enquanto a atual está esperando – por isso é possível disparar os diálogos do Mago e do Elfo juntos, sem que um precise esperar o outro terminar por completo.

4.3 Por que foi necessário um semáforo

Mesmo com tasks assíncronas, o `Console` é um recurso compartilhado. Sem sincronização, duas tasks poderiam escrever no console ao mesmo tempo e misturar os caracteres das falas. O `SemaphoreSlim` resolve isso garantindo exclusão mútua durante a escrita de cada fala, sem bloquear a thread (porque `WaitAsync` também é assíncrono).

5 Resultado obtido

A saída abaixo foi gerada executando `dotnet run` e corresponde ao resultado esperado no enunciado:

```
Mago: Quem é você viajante ?...  
Elfo: Sou link do jogo zelda...  
Mago: OK, sou Gandalf do senhor dos anéis...  
Mago: Se diz ser quem é, então diga as palavras secretas comigo...  
Mago: E as portas se abrirão...  
Mago: Anah Elfo: Milac Mago: Passi-cii-fiik Elfo: Teish Mago: Perim Elfo: Salaa-saa-aah  
  
*****  
Portas abertas!
```

6 Conclusão

O trabalho permitiu observar, na prática, a diferença entre execução síncrona (bloqueante) e assíncrona (não bloqueante), além do uso de **Task** para paralelizar o diálogo de dois personagens que, mesmo “conversando ao mesmo tempo”, precisam de sincronização (semáforo) para que a saída no console permaneça legível e organizada – exatamente o comportamento clássico de concorrência sobre um recurso compartilhado.